

1. Why is Inference Memory-Bandwidth Bound Important?

When a model is trained, it learns a large number of parameters or weights. During **inference**, we are not training the model; instead, we use these already-learned weights to generate an output.

For example, when we give a prompt to an LLM, the model processes the prompt and generates the answer. In an autoregressive LLM, tokens are generally generated one at a time. For every new token, the model performs many calculations using its weights.

The important point is that the GPU must **continuously read data from memory** while performing these calculations. Modern GPUs can perform calculations extremely quickly, but moving data between memory and the GPU's compute units can become the main limitation.

This is where **memory bandwidth** becomes important. Memory bandwidth describes how quickly data can be transferred between the GPU's memory and its compute units. It is usually measured in **GB/s or TB/s**.

For example, if a model has several gigabytes of weights, those weights need to be accessed repeatedly during inference. If the GPU can perform calculations faster than it can receive the required data, the compute units may remain idle while waiting for memory.

In this situation, the inference workload is called **memory-bandwidth bound**.

A simple way to understand it is:

The GPU may be able to calculate very quickly, but it still has to wait for the data needed for those calculations.

This is especially important for LLM decoding because tokens are generated sequentially. When only a small number of requests are being processed at the same time, there may not be enough parallel work to fully use the GPU's computing power.

There are two common types of bottlenecks:

- **Compute-bound:** The GPU spends most of its time performing mathematical calculations.
- **Memory-bandwidth-bound:** The GPU spends more time waiting for data to be transferred from memory.

2. How Can We Calculate the Overall Memory Required to Store a Model? How Are LLMs Represented When Stored?

The storage size of an LLM mainly depends on two things: **the total number of parameters** and **the precision used to represent those parameters**.

Parameters are the learned numerical values that contain the information obtained by the model during training. Since an LLM can contain billions of parameters, the choice of numerical format has a major effect on the final model size.

Some commonly used formats are:

- **FP32** $\rightarrow$ 32 bits = 4 bytes
- **FP16** $\rightarrow$ 16 bits = 2 bytes
- **BF16** $\rightarrow$ 16 bits = 2 bytes
- **INT8** $\rightarrow$ 8 bits = 1 byte
- **INT4** $\rightarrow$ 4 bits = 0.5 byte

The basic calculation can be written as:

$$\text{Model Memory} = \text{Number of Parameters} \times \text{Bytes per Parameter}$$

For instance, consider a model containing **7 billion parameters**.
With FP32:

$$7 \times 10^9 \times 4 = 28 \times 10^9 \text{ bytes}$$

So the weights would need roughly **28 GB** of storage.
With FP16:

$$7 \times 10^9 \times 2 = 14 \times 10^9 \text{ bytes}$$

This reduces the size to approximately **14 GB**.

If the model is quantized to INT8, the requirement becomes about **7 GB**, and with INT4 it becomes about **3.5 GB**. Therefore, using lower-precision formats can significantly reduce the memory needed to store a model.

How is an LLM stored internally?

An LLM is not saved as a single continuous collection of numbers. Instead, its parameters are divided into many **tensors**. A tensor is essentially a structured collection of numerical values, which can have one or more dimensions.

For a Transformer-based model, different tensors are responsible for different parts of the network. They may represent:

- embedding values,
- attention parameters,
- query, key, and value matrices.

3. Explain Each Layer of Inferencing and Its Importance to a Company

LLM inference can be understood through three main layers: **runtime**, **infrastructure**, and **tooling**. Each layer has a different role, but together they help a company serve AI models efficiently and reliably.

3.1 Runtime

The **runtime** is the software layer that actually runs the LLM. It loads the model and performs the computations required to generate responses.

Some examples are **vLLM**, **TensorRT-LLM**, and **llama.cpp**.

The runtime is responsible for:

- loading model weights,
- running model computations,
- generating tokens,
- managing the KV cache,
- scheduling and batching requests,
- communicating with the available hardware.

The main goal of the runtime is to **run the model as efficiently as possible**. A well-optimized runtime can reduce response time and increase the number of requests handled by the same GPU. For a company, this can directly reduce infrastructure and inference costs.

3.2 Infrastructure

Infrastructure refers to the hardware and supporting systems used to run the inference workload.

It includes:

- GPUs or other accelerators,
- CPUs and system memory,
- storage and networking,
- servers and clusters,
- load balancing and autoscaling.

For example, if a company has many users accessing an LLM at the same time, several GPUs or servers may need to work together. Good infrastructure allows the system to **handle increasing traffic while maintaining performance and reliability**.

It also helps companies use their hardware efficiently and avoid unnecessary operating costs.

3.3 Tooling

Tooling includes the additional software used to develop, monitor, optimize, and manage the inference system.

It can include tools for:

- monitoring GPU usage and latency,
- measuring throughput,
- profiling performance,
- logging errors,
- managing deployments,
- testing different model configurations.

Tooling is important because simply running a model is not enough for a production system. Companies need to know **how the system is performing and where bottlenecks are occurring**.

Overall Importance

The three layers have different responsibilities:

Runtime executes the model, infrastructure provides the computing resources, and tooling helps monitor and improve the system.

The specific technologies used in these layers can change, so they are **not irreplaceable**. What is important is that all three functions are properly covered. Together, they help a company achieve **better performance, scalability, reliability, and lower inference costs**.